ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.
Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

ASSIGNMENT 2

Subject Name: Exploratory Data Analysis
Subject Code: BAI515E

Submitted By:
VINAY KUMAR
1AY25AI412

Submitted To:
Ms. Surbhi

TABLE OF CONTENTS

Sl. No.	Section	Page
1	Introduction and dataset overview	4
2	Loading and inspecting the data	4
3	Decade totals and gender comparison	5
4	Robust cleaning with sigma-clipping	7
5	Date index and weekday patterns	8
6	Birth counts by calendar date	10-11
7	Pivot table and groupby comparison	12
8	Bonus: dates with unusually low birth counts	13-14
9	Conclusion	15
10	References	15

LIST OF FIGURES

Sl. No.	Figure title	Page
1	Birth totals across decade groups	5
2	Relative gender difference by decade	6
3	Average daily births by weekday and decade (heatmap)	9
4	Calendar-day pattern with a smoothed trend	11
5	Ten dates with the lowest average birth counts	14

LIST OF TABLES

Sl. No.	Table title	Page
1	Raw dataset summary	4
2	Total births by decade and gender	5
3	Male-to-female ratios by decade	6
4	Robust clipping statistics	7
5	Mean daily births by weekday and decade	9
6	Timing comparison template	12
7	Lowest ten calendar dates by mean births	13

1.INTRODUCTION

Exploratory Data Analysis helps reveal patterns, unusual records and relationships in a dataset before more advanced modelling is attempted. In this assignment, the data consists of aggregated births in the United States, recorded by year, month, day, gender and birth count. Python libraries are used to examine changes across decades, weekdays and calendar dates.

The report uses Pandas for table operations, NumPy for robust statistics, and Matplotlib for plots. The work begins with a copy of the raw data so that totals based on the original file can be compared with summaries from the cleaned daily records. The cleaning stage is important because some records contain incomplete or invalid day values and unusually small or large counts.

2. LOADING AND INSPECTING THE DATA

The CSV file is read from its public source URL. A separate copy is retained before cleaning because decade totals should include the full period represented in the original data, including records without a daily date after 1988.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_URL = 'https://raw.githubusercontent.com/jakevdp/data-CDCbirths/master/births.csv'
births = pd.read_csv(DATA_URL)
births_raw = births.copy()

print(births.head())
print('Shape:', births.shape)
print(births.dtypes)
print('Missing values:')
print(births.isna().sum())
```

Property	Observed value	Meaning
Rows and columns	(15547, 5)	15,547 records and five fields in the raw file.
Columns	year, month, day, gender, births	Calendar fields, group label and count.
Missing values in day	480	The day is not supplied for part of the source period.
Missing in other fields	0 each	The other columns have no missing values in this dataset.

Table 1: Initial row count, fields and missing-value summary.

The day column is stored as a floating-point value because missing entries are represented by NaN. This allows the same column to contain ordinary day numbers and missing values. It should not be forced to integer type until incomplete records have been filtered out.

```
# Keep a labelled copy for the decade summary before cleaning
births_raw['decade'] = (births_raw['year'] // 10) * 10
print(births_raw[['year', 'month', 'day', 'gender', 'births']].head())
```

3. DECADE TOTALS AND GENDER COMPARISON

A decade label is calculated by integer-dividing each year by ten and multiplying the result by ten. The pivot table places decades in rows and the two reported gender categories in columns. The aggregation adds the birth counts falling in each decade group.

```
decade_gender = births_raw.pivot_table(
    values='births', index='decade', columns='gender', aggfunc='sum'
)
print(decade_gender)
print('Male count exceeds female count in every group:',
      (decade_gender['M'] > decade_gender['F']).all())
```

Decade label	Female births	Male births
1960	1,753,634	1,846,572
1970	16,263,075	17,121,550
1980	18,310,351	19,243,452
1990	19,479,454	20,420,553
2000	18,229,309	19,106,428

Table 2: Sum of births by decade label and gender.

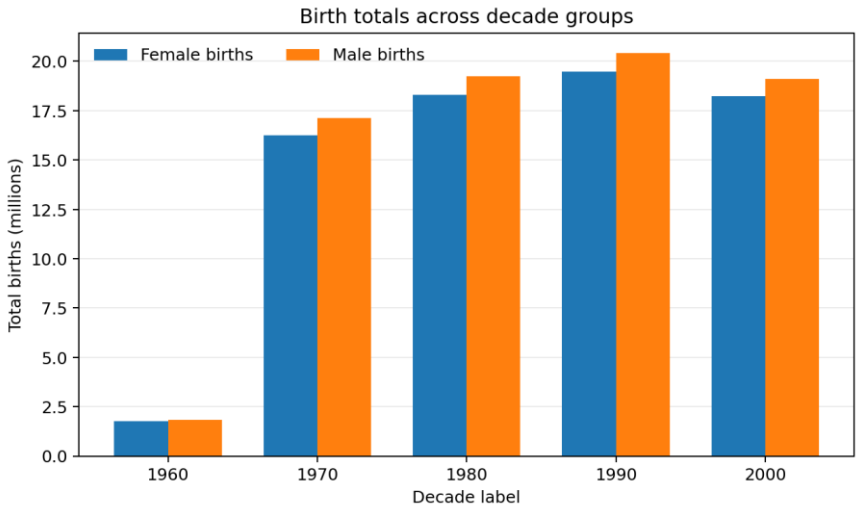


Figure 1: Birth totals across decade groups, displayed with grouped bars.

The male total is higher than the female total in each decade group. The 1960 group represents only 1969, while the 2000 group spans 2000-2008 in this file. For that reason, total heights should be compared within each decade rather than treating the first and last groups as full ten-year periods.

3.1 Ratio and percentage difference

```
ratio_table = pd.DataFrame(index=decade_gender.index)
ratio_table['M/F ratio'] = decade_gender['M'] / decade_gender['F']
ratio_table['Male excess (%)'] = (ratio_table['M/F ratio'] - 1) * 100
print(ratio_table.round(4))
```

Decade	M/F ratio	Male excess over female (%)
1960	1.0530	5.30%
1970	1.0528	5.28%
1980	1.0510	5.10%
1990	1.0483	4.83%
2000	1.0481	4.81%

Table 3: Relative difference between male and female totals.

```
ax = ratio_table['Male excess (%)'].plot(kind='bar', figsize=(7, 3.5))
ax.set_xlabel('Decade label')
ax.set_ylabel('Male excess (%)')
ax.set_title('Relative difference in totals by decade')
plt.tight_layout()
plt.show()
```

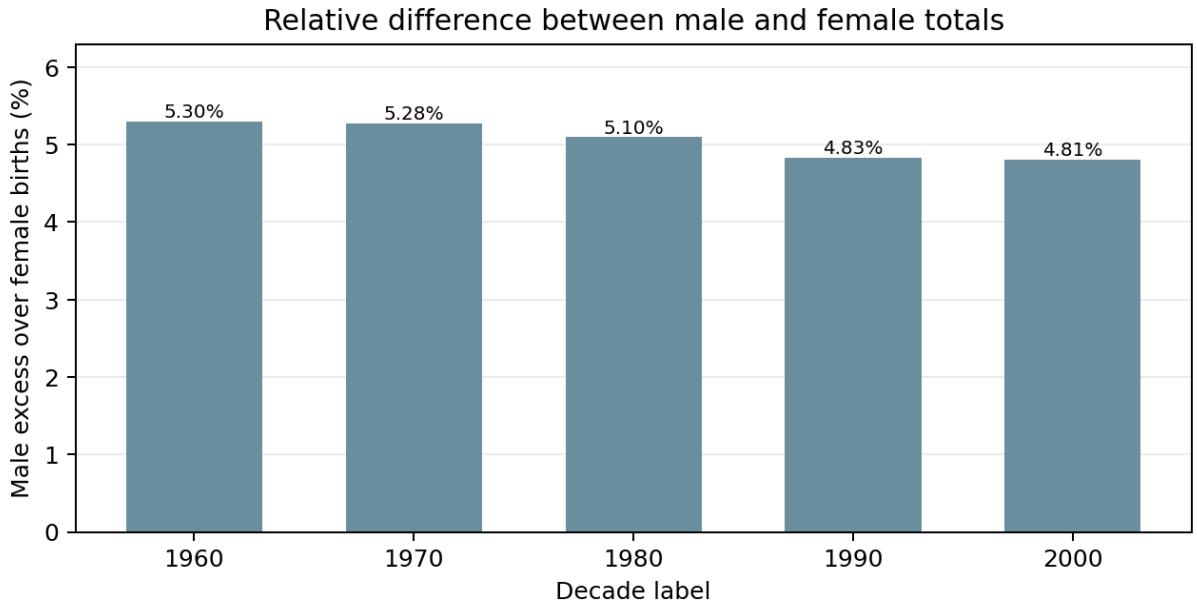


Figure 2: Percentage by which male births exceed female births.

The ratio stays close to 1.05. Expressed as a percentage, male births are roughly 4.8% to 5.3% higher than female births across these groups. The ratio provides a fairer comparison than the raw difference because the decade groups contain different numbers of years.

4. ROBUST CLEANING WITH SIGMA-CLIPPING

The original table includes entries that cannot represent a normal daily date, along with records where a day value is missing. The associated counts differ greatly from usual daily counts. A robust range is estimated from the median and interquartile range, which are less influenced by extreme values than the ordinary mean and standard deviation.

4.1 Robust centre and spread

```
q1, median, q3 = np.percentile(births['births'], [25, 50, 75])
mu = median
sigma = 0.74 * (q3 - q1)
low_limit = mu - 5 * sigma
high_limit = mu + 5 * sigma

print('Q1:', q1, 'Median:', median, 'Q3:', q3)
print('Robust sigma:', round(sigma, 2))
print('Accepted interval:', round(low_limit, 1), 'to', round(high_limit, 1))
```

Statistic	Value	Explanation
First quartile (Q1)	4,358.0	25% of values lie at or below this point.
Median (mu)	4,814.0	Robust centre of the birth-count distribution.
Third quartile (Q3)	5,289.5	75% of values lie at or below this point.
Robust sigma	689.31	0.74 multiplied by the interquartile range.
Keep interval	1,367.5 to 8,260.5	Counts inside this interval are retained.

Table 4: Robust estimates and the five-sigma acceptance interval.

4.2 Apply the filter

```
rows_before = len(births)
births = births.loc[
    (births['births'] > low_limit) & (births['births'] < high_limit)
].copy()
print('Rows before:', rows_before)
print('Rows retained:', len(births))
print('Rows removed:', rows_before - len(births))

births['day'] = births['day'].astype(int)
print('Missing cells remaining:', int(births.isna().sum().sum()))
```

For this dataset, the reported filtering output retains 14,610 records from the original 15,547 and removes 937. Once these abnormal and incomplete entries are excluded, the day column can be converted to an integer. The median and IQR are selected because extreme values would otherwise influence the mean and standard deviation used to define the cut-off.

This is a practical cleaning rule rather than proof that every removed observation is an error in every possible dataset. The limits are suitable here because the assignment data contains very small counts from malformed day entries and very large aggregated counts where only a month was recorded.

5. DATE INDEX AND WEEKDAY PATTERNS

5.1 Build a datetime index

Pandas can combine the year, month and day fields into one timestamp for each retained record. The datetime index then provides weekday information directly. A decade field is also added to the cleaned dataset because it is used in the weekday comparison.

```
births.index = pd.to_datetime(births[['year', 'month', 'day']])
births['dayofweek'] = births.index.dayofweek  # Monday=0 ... Sunday=6
births['decade'] = (births['year'] // 10) * 10
print(births.head())
print('Date range:', births.index.min().date(), 'to', births.index.max().date())
```

The cleaned dates range from 1 January 1969 to 31 December 1988. This leaves three decade labels in the daily analysis: 1960, 1970 and 1980. Records from later years are absent because the source does not provide an individual day for that period.

5.2 Mean birth counts by weekday

```
weekday_summary = births.pivot_table(
    values='births', index='dayofweek', columns='decade', aggfunc='mean'
)
weekday_summary = weekday_summary.reindex(range(7))
print(weekday_summary.round(1))

# Heatmap: cells show the mean daily count for each weekday and decade.
plt.figure(figsize=(8, 4.8))
plt.imshow(weekday_summary.T, aspect='auto', cmap='YlGnBu')
plt.xticks(range(7), ['Mon','Tue','Wed','Thu','Fri','Sat','Sun'])
plt.yticks(range(len(weekday_summary.columns)), weekday_summary.columns)
plt.xlabel('Day of week')
plt.ylabel('Decade')
plt.title('Average daily births by weekday and decade')
plt.colorbar(label='Mean births')
plt.tight_layout()
plt.show()
```

Weekday	1960 group	1970 group	1980 group
Mon	5063.8	4689.1	5276.9
Tue	5286.1	4885.3	5503.8
Wed	5074.6	4750.4	5367.6
Thu	4978.3	4696.9	5333.5
Fri	5107.9	4782.1	5393.1
Sat	4651.1	4207.8	4483.9
Sun	4342.3	3979.3	4308.1

Table 5: Mean daily births grouped by weekday and decade label.

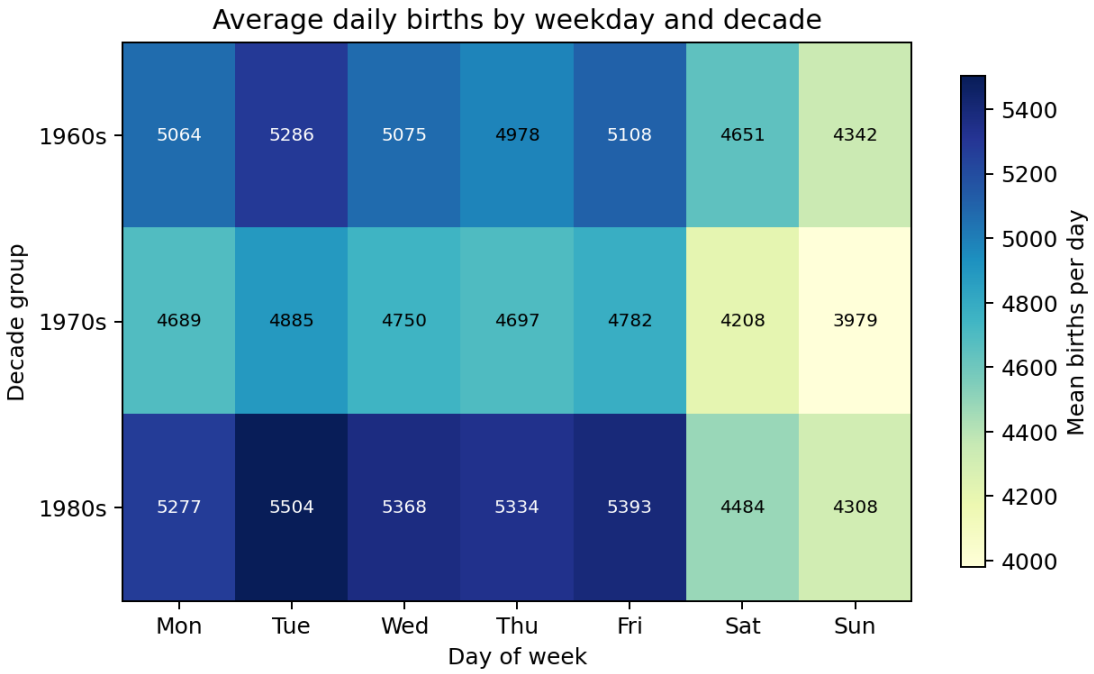


Figure 3: Heatmap of the weekday averages. Darker cells represent higher averages.

In each of the three decade groups, the Saturday and Sunday averages are lower than the weekday values. Tuesday is the highest weekday in all three groups. One likely explanation is scheduling: planned inductions and caesarean deliveries are more often arranged on weekdays than weekends. The pattern should not be interpreted as proof that natural birth rates follow the same weekly cycle.

6. BIRTH COUNTS BY CALENDAR DATE

6.1 Create the calendar-date summary

To compare the same date across different years, the cleaned records are grouped by month and day and their birth counts are averaged. A leap year is used for the plotting index so that 29 February can be represented without dropping the date.

```
by_calendar_date = births.pivot_table(
    values='births', index=['month', 'day'], aggfunc='mean'
)
parts = by_calendar_date.index.to_frame(index=False)
parts['year'] = 2012  # a leap year includes February 29
by_calendar_date.index = pd.to_datetime(parts[['year', 'month', 'day']])
by_calendar_date = by_calendar_date.sort_index()
by_calendar_date.columns = ['mean_births']
print('Number of calendar dates:', len(by_calendar_date))
print(by_calendar_date.head())
```

Using a fixed leap year turns month-day pairs into a continuous time axis. In this analysis the calendar summary contains 366 dates. A rolling average is added to the plot to make the broad seasonal movement easier to see while the thinner daily line preserves short-term changes.

6.2 Plot the daily pattern

```
smooth = by_calendar_date['mean_births'].rolling(7, center=True, min_periods=1).mean()
fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(by_calendar_date.index, by_calendar_date['mean_births'],
        linewidth=0.8, alpha=0.55, label='Calendar-date mean')
ax.plot(smooth.index, smooth, linewidth=2, label='7-day rolling average')
for day, label in [('2012-01-01', "New Year's Day"),
                   ('2012-07-04', 'July 4'), ('2012-12-25', 'Christmas')]:
    stamp = pd.Timestamp(day)
    ax.annotate(label, xy=(stamp, by_calendar_date.loc[stamp, 'mean_births']),
                xytext=(6, 10), textcoords='offset points', fontsize=8,
                arrowprops=dict(arrowstyle='->', linewidth=0.7))
ax.set_xlabel('Calendar date (leap year 2012)')
ax.set_ylabel('Mean births')
ax.set_title('Calendar-day pattern with a smoothed trend')
ax.legend()
ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

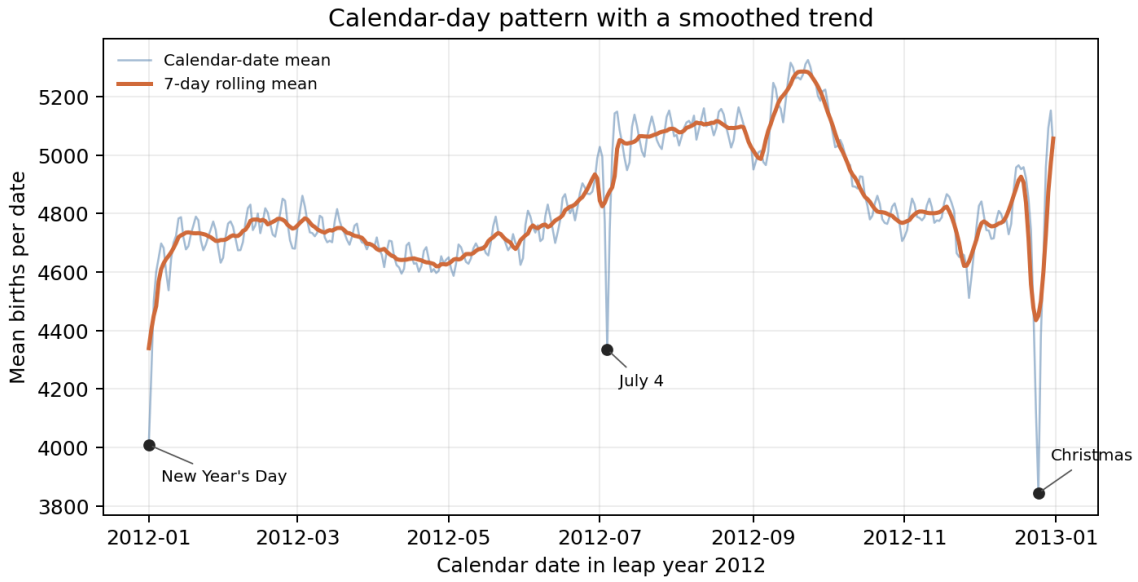


Figure 4: Daily calendar-date averages and a seven-day moving trend.

The calendar curve is not flat throughout the year. Average counts rise through summer, are comparatively high around early autumn, and drop at several holidays. Individual dates can stand out sharply because planned deliveries are less frequent on major holidays. The chart is descriptive: the pattern suggests an association with scheduling rather than establishing a medical cause.

7. PIVOT TABLE AND GROUPBY COMPARISON

7.1 Recreate the decade table with groupby

A pivot table is convenient for arranging categories across rows and columns. The same decade-by-gender result can be constructed by grouping on both keys, summing the counts and using unstack to move the gender key into columns.

```
by_group = (births_raw.groupby(['decade', 'gender'])['births']
            .sum().unstack())
print(by_group)
print('Same values as pivot_table:', by_group.equals(decade_gender))
pd.testing.assert_frame_equal(by_group, decade_gender, check_names=False)
```

The equality check should return True because the two expressions perform the same grouping and aggregation over the same uncleaned data. Using births_raw is important here: the comparison should not mix a raw-data table with a cleaned-data table.

7.2 Compare execution time

The two methods can be timed in a Jupyter notebook using %timeit. Repeated measurements are preferred over one run because background activity, caching and the computer configuration affect small timing differences.

```
def build_pivot():
    return births_raw.pivot_table(values='births', index='decade',
                                  columns='gender', aggfunc='sum')

def build_groupby():
    return births_raw.groupby(['decade', 'gender'])['births'].sum().unstack()

pivot_time = %timeit -o build_pivot()
group_time = %timeit -o build_groupby()
print('pivot_table mean (ms):', pivot_time.average * 1000)
print('groupby + unstack mean (ms):', group_time.average * 1000)
print('pivot/groupby ratio:', pivot_time.average / group_time.average)
```

Method	Mean time (ms)	Standard deviation (ms)	Relative result
pivot_table	[enter %timeit result]	[enter %timeit result]	Compare to groupby
groupby + unstack	[enter %timeit result]	[enter %timeit result]	Baseline

Table 6: Fill these cells with the timings reported by your own Jupyter run.

Both approaches separate observations into groups and apply an aggregation. pivot_table offers a concise spreadsheet-like layout and additional options such as margins and fill values. groupby is more general-purpose and can be easier to extend to other aggregations. In many cases groupby followed by unstack can be slightly faster, but the result should be determined by measurement rather than assumed.

8. BONUS: DATES WITH THE LOWEST AVERAGE BIRTH COUNTS

The nsmallest method returns the calendar dates with the lowest mean count without requiring a manual sort of the full series. A small dictionary maps selected month-day combinations to fixed-date holidays or nearby dates so the results can be inspected in context.

```
lowest10 = by_calendar_date['mean_births'].nsmallest(10)
print(lowest10.round(1))

holiday_names = {
    (1, 1): "New Year's Day", (1, 2): 'Day after New Year',
    (7, 4): 'Independence Day', (12, 24): 'Christmas Eve',
    (12, 25): 'Christmas Day', (12, 26): 'Day after Christmas'
}
labels = []
for date in lowest10.index:
    labels.append(holiday_names.get((date.month, date.day), 'No fixed-date holiday'))

low_table = lowest10.rename('mean_births').to_frame()
low_table['calendar note'] = labels
print(low_table.round(1))
```

Rank	Date	Mean births	Calendar note
1	December 25	3,844.4	Christmas Day
2	January 01	4,009.2	New Year's Day
3	December 24	4,126.2	Christmas Eve
4	January 02	4,247.4	Day after New Year
5	July 04	4,335.3	Independence Day
6	December 26	4,383.5	Day after Christmas
7	December 23	4,466.7	Christmas Eve period
8	January 03	4,500.9	Early January
9	November 27	4,511.6	No fixed-date holiday
10	January 09	4,537.8	No fixed-date holiday

Table 7: Ten calendar dates with the lowest reported mean birth counts.

8.1 Visual comparison of the lowest dates

```
plot_data = low_table.sort_values('mean_births')
ax = plot_data['mean_births'].plot(kind='barh', figsize=(8, 5))
ax.set_xlabel('Mean births')
ax.set_ylabel('Calendar date')
ax.set_title('Ten dates with the lowest average birth counts')
plt.tight_layout()
plt.show()
```

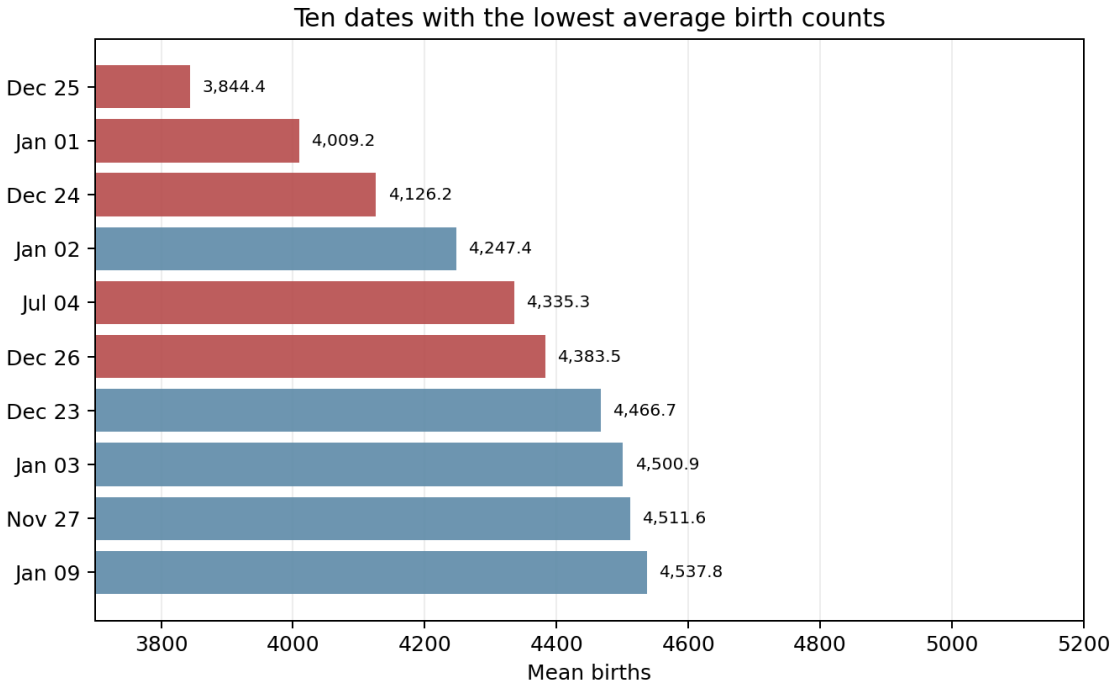


Figure 5: Horizontal ranking of the ten lowest calendar-date means.

Christmas Day and New Year's Day are among the lowest values in this summary, with Christmas Day producing the smallest mean. July 4 also appears in the lower group. The adjacent holiday dates are low as well, which is consistent with planned procedures being shifted away from public holidays. Some low-count dates do not match a fixed-date holiday, so the holiday explanation should be treated as a pattern to investigate rather than a complete explanation for each point.

9. CONCLUSION

The analysis used Pandas and NumPy to inspect birth-count records, group totals by decade and gender, and clean unusual entries with a robust five-sigma rule. Male totals were higher than female totals in each decade group, while the male-to-female ratio remained close to 1.05. Decade totals need context because the first and final groups cover partial decades.

After cleaning, the daily data covered 1969 through 1988. The weekday summary showed lower counts on Saturday and Sunday, with Tuesday having the highest average among the days shown. Averaging by month and day revealed several low dates around New Year, Independence Day and Christmas. These patterns may reflect how planned deliveries are scheduled, but the analysis is descriptive and cannot by itself prove causation.

Finally, pivot_table and groupby with unstack produced the same decade-by-gender summary. Their run times should be measured on the computer used for the assignment. The report demonstrates that meaningful EDA depends not only on plotting but also on checking missing values, preserving the raw data, using an appropriate cleaning rule, and explaining limitations in the results.

10. REFERENCES

1. Jake VanderPlas, data-CDCbirths repository, aggregated US births dataset. https://github.com/jakevdp/data-CDCbirths
2. Pandas Development Team, pandas documentation: pivot_table, groupby, datetime indexing and rolling windows. https://pandas.pydata.org/docs/
3. NumPy Developers, NumPy documentation: percentiles and numerical arrays. https://numpy.org/doc/
4. Matplotlib Development Team, Matplotlib documentation: bar charts, image plots, annotations and line plots. https://matplotlib.org/stable/